



Employee Login & Registration System

Objective

Create a simple console application where employees can register and login

1 CORE REQUIREMENTS

1.1 What Your Program Should Do

1. **Register New Employee** - Collect and store employee details
2. **Login Existing Employee** - Verify credentials and show profile
3. **View All Employees** - Display registered employees (optional)

1.2 The Employee Class

Create a class called Employee with these properties:

- Name (string)
- Surname (string)
- Age (int)
- Salary (decimal)
- Role (string - like "Manager", "Developer", "Sales")
- Username (string)
- Password (string - private field)
- Employee ID (int)

1.3 Program Features

1.3.1 Main Menu

```
=== EMPLOYEE SYSTEM ===  
  
1. Register New Employee  
2. Login  
3. View All Employees  
4. Exit
```

1.3.2 Registration Flow

```
Enter your name: John
Enter your surname: Doe
Enter your age: 28
Enter your salary: 45000.50
Enter your role: Developer
Create username: johndoe
Create password: *****

Employee registered successfully!
Your Employee ID: 1001
```

1.3.3 Login Flow

```
Enter username: johndoe
Enter password: *****
=== LOGIN SUCCESSFUL ===
Welcome back, John Doe!

Your Profile:
- Employee ID: 1001
- Name: John Doe
- Age: 28
- Role: Developer
- Salary: $45,000.50
```

2 TECHNICAL CONCEPTS TO DEMONSTRATE

2.1 Data Types

- string - for name, surname, role, username
- int - for age, employee ID
- decimal - for salary (money)
- bool - for login status, active account

2.2 Access Modifiers

- public - properties that need to be accessed (Name, Surname, etc.)
- private - sensitive data (password field)
- internal - for the Employee class itself
- private - for Employee ID

2.3 Storage

- Use an ArrayList or List<Employee> to store registered employees
- Store in memory only (no files or databases)

2.4 if Statements

You'll need if statements for:

1. Checking if username already exists during registration
2. Validating age (must be 18 or older)
3. Verifying password during login
4. Checking if employee found in ArrayList
5. Validating inputs aren't empty

3 COMPLETE CHECKLIST

3.1 Data Types Used

- string (name, surname, role, username)
 - int (age, employee ID)
 - decimal (salary)
-

- bool (optional for active status)

3.2 Access Modifiers Used

- public (properties and methods)
- private (password field)
- internal (class)

3.3 if Statements Used

- Age validation
- Empty input validation
- Username uniqueness check
- Login credential verification
- Salary validation

3.4 ArrayList Operations

- Add employees to ArrayList
- Search through ArrayList
- Display all employees

3.5 Sample Test Cases

1. **Register** a new employee
2. Try to register with same username - should show error
3. Try to register with age 17 - should show error
4. **Login** with correct credentials - should show profile
5. Login with wrong password - should show error
6. Login with non-existent username - should show error
7. View all employees - should list everyone

3.6 What to Submit

1. Complete C# code (Employee class and Program class)
2. Brief explanation covering:
 - What data types you used and why

- Examples of your if statements
- What you made public vs private and why
- One challenge you faced